INSIGNIA

AWS Business Unit

Data Engineer

Take-Home Assessment

Level - 3 (Senior)

Prepared by: Data Engineering Team

AWS Business Unit

Version 2.0 | February 2026

General Instructions

Welcome to the Insignia Data Engineer technical assessment. This take-home test evaluates your practical skills across three core competencies: data pipeline development (Python/PySpark), SQL query design and optimization, and data architecture and system design.

Important: Read the scenario carefully before starting — the context you build in Question 1 carries through to Questions 2 and 3.

Evaluation Criteria

• Pipeline Design: Clean, testable, production-ready ETL/ELT code with proper error handling and logging.

• SQL Proficiency: Correct, performant queries demonstrating window functions, CTEs, and optimization awareness.

• Architecture Thinking: Understanding of data warehouse design, medallion architecture, scalability, and trade-offs.

• Explanation: Clear reasoning behind your approach. We value your thought process as much as the solution.

LEVEL 3 — SENIOR DATA ENGINEER

This level tests advanced architectural thinking, cross-team leadership, and enterprise-scale engineering skills expected of a senior data engineer. All three questions are set within the same company and dataset — read the scenario first.

📖 THE SCENARIO — MegaMart Indonesia

MegaMart Indonesia is the country's largest omnichannel retail conglomerate with 850+ stores across 34 provinces, 12 million loyalty members, and $2.8B annual GMV. They operate multiple formats:

- MegaMart Hypermarket (120 stores) — Large-format stores
- MegaMart Express (580 stores) — Convenience stores in urban areas
- MegaMart Online (e-commerce) — 2M+ daily orders
- MegaMart Wholesale (45 stores) — B2B bulk sales
- MegaMart Fresh (105 dark stores) — Grocery delivery-only

Business Context

MegaMart is undergoing a digital transformation. The CEO has mandated:

- Unified Customer 360 — Single view of customer across all channels
- Inventory Optimization — Daily stock visibility across all 850 stores
- Regulatory Compliance — Full audit trail for tax authority (DJP)

Technical Challenges

- Data Silos: 5 separate Snowflake accounts, 3 on-prem Oracle instances
- Inconsistent Definitions: "Active Customer" has 7 different definitions
- Scale: Daily batch processing of 200M+ records
- Legacy Debt: 200+ Airflow DAGs, 40% failure rate
- Talent Gap: Junior team of 8 data engineers

Database Schema

bronze.transactions

Column	Type	Description
txn_id	VARCHAR(30)	Unique transaction ID
store_id	VARCHAR(15)	Store identifier
customer_id	VARCHAR(20)	FK to customers (nullable)
txn_timestamp	TIMESTAMP	Transaction time
channel	VARCHAR(20)	IN_STORE, ONLINE, MARKETPLACE, WHOLESALE
sub_channel	VARCHAR(30)	HYPERMARKET, EXPRESS, TOKOPEDIA, etc.
total_amount	DECIMAL(15,2)	Gross transaction amount
discount_amount	DECIMAL(15,2)	Total discounts applied
tax_amount	DECIMAL(15,2)	VAT (11%)
payment_method	VARCHAR(20)	CASH, CARD, EWALLET, QRIS

bronze.customers

Column	Type	Description
customer_id	VARCHAR(20)	Unique customer ID
full_name	VARCHAR(150)	Customer name
nik	VARCHAR(16)	National ID (KTP)
phone	VARCHAR(20)	Primary phone
email	VARCHAR(100)	Email address
city	VARCHAR(50)	City
province	VARCHAR(50)	Province
registration_date	DATE	Loyalty signup date
loyalty_tier	VARCHAR(15)	BRONZE, SILVER, GOLD, PLATINUM, DIAMOND
lifetime_value	DECIMAL(15,2)	Calculated LTV

bronze.inventory_daily

Column	Type	Description
snapshot_date	DATE	Date of inventory snapshot
store_id	VARCHAR(15)	Store/warehouse ID
sku_id	VARCHAR(20)	FK to products
opening_stock	DECIMAL(10,3)	Stock at start of day
received	DECIMAL(10,3)	Units received
sold	DECIMAL(10,3)	Units sold
waste	DECIMAL(10,3)	Units wasted/expired
closing_stock	DECIMAL(10,3)	Stock at end of day

Question 1: Platform Consolidation — Unifying 5 Data Silos

PySpark, Data Integration, Schema Evolution, CDC, Data Quality

Context

MegaMart's 5 business units each have their own Snowflake account with different schemas. The CEO wants a unified data platform within 3 months.

Current State:

• Hypermarket: hypermart_prod.raw.transactions — 8M rows/day

• Express: express_prod.raw.sales — 12M rows/day

• Online: ecomm_prod.raw.orders — 2M rows/day

• Wholesale: wholesale_prod.raw.invoices — 500K rows/day

• Fresh: fresh_prod.raw.deliveries — 1M rows/day

Given Code (Refactor This)

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("consolidate").getOrCreate()

def load_all_sources():
    hyper =
spark.read.format("snowflake").options(**hyper_config).option("dbtable",
"raw.transactions").load()
    express =
spark.read.format("snowflake").options(**express_config).option("dbtable",
"raw.sales").load()
    online =
spark.read.format("snowflake").options(**online_config).option("dbtable",
"raw.orders").load()
    wholesale =
spark.read.format("snowflake").options(**wholesale_config).option("dbtable",
"raw.invoices").load()
    fresh =
spark.read.format("snowflake").options(**fresh_config).option("dbtable",
"raw.deliveries").load()

    all_data = hyper.union(express).union(online).union(wholesale).union(fresh)
    all_data.write.mode("overwrite").parquet("s3://megamart-
unified/transactions/")
    print("Done!")

load_all_sources()
```

Tasks

Task 1a: Problem Identification

Identify at least 8 critical problems with the given pipeline. For each: explain production impact, categorize (Performance/Correctness/Reliability/Security/Maintainability), and propose fix.

Task 1b: Schema Harmonization Design

Design a schema harmonization framework: mapping specification (table format), PySpark code for one source transformation, and schema registry design for tracking source-to-target mappings.

Task 1c: Incremental CDC Pipeline

Rewrite as production-grade incremental batch pipeline: watermark-based loading, late-arriving data handling (7 days), MERGE/upsert logic, idempotency, dead-letter queue, logging and metrics.

Task 1d: Technical Design Document

Create 1-2 page design doc for mid-level engineers: architecture decisions, how to add new source, troubleshooting guide, monitoring approach.

Question 2: Advanced Analytical Pipelines — Customer 360 & Inventory

PySpark, SQL, Data Modeling, SCD Type 2, Performance Optimization

Context

With the unified Bronze layer from Q1, build Silver and Gold layers for business analytics.

Required Formulas

Use these exact formulas in your calculations:

Customer Metrics:

• total_transactions = COUNT(DISTINCT txn_id) per customer

• total_spend = SUM(total_amount - discount_amount) per customer

• avg_basket_size = total_spend / total_transactions

• days_since_last_purchase = CURRENT_DATE - MAX(txn_timestamp)::DATE

• days_since_registration = CURRENT_DATE - registration_date

• avg_monthly_spend = total_spend / GREATEST(MONTHS_BETWEEN(CURRENT_DATE, first_purchase_date), 1)

• preferred_channel = channel with MAX transaction count

• preferred_category = category_l1 with MAX spend

Spend Trend Classification:

• Compare last 3 months avg spend vs. previous 3 months avg spend

• INCREASING: last_3m_avg > prev_3m_avg * 1.1

• DECREASING: last_3m_avg < prev_3m_avg * 0.9

• STABLE: otherwise

Inventory Metrics:

• days_of_stock = closing_stock / NULLIF(avg_daily_sales_7d, 0)

• avg_daily_sales_7d = AVG(sold) OVER (PARTITION BY store_id, sku_id ORDER BY snapshot_date ROWS 6 PRECEDING)

• waste_rate = waste / NULLIF(opening_stock + received, 0)

• stock_turnover = SUM(sold) over 30 days / AVG(closing_stock) over 30 days

• inventory_balance_check: closing_stock = opening_stock + received - sold - waste

Alert Flags:

• out_of_stock_flag = TRUE if closing_stock <= 0

• overstock_flag = TRUE if days_of_stock > 60

• high_waste_flag = TRUE if waste_rate > 0.10 (10%)

• reorder_point_flag = TRUE if days_of_stock < 7

Tasks

Task 2a: Customer 360 Pipeline

Build daily batch pipeline for gold.dim_customer_360 using the formulas above. Write PySpark code with proper partitioning, handle customers with no transactions (set metrics to 0 or NULL as appropriate), include DQ checks.

Task 2b: SCD Type 2 for Customer Dimension

Implement SCD Type 2 for dim_customers (track loyalty_tier and city changes):

• DDL with: surrogate_key, customer_id (natural key), attributes, effective_date, expiry_date (9999-12-31 for current), is_current (1/0)

• PySpark MERGE logic: new customers (INSERT), changed attributes (UPDATE old row's expiry_date, INSERT new row), unchanged (skip)

• SQL query: "What was customer's loyalty_tier at the time of a specific transaction?" (JOIN on customer_id WHERE txn_date BETWEEN effective_date AND expiry_date)

Task 2c: Inventory Health Pipeline

Build daily batch pipeline for gold.inventory_health using the formulas above. Write SQL with window functions. Optimize for 850 stores × 50,000 SKUs = 42.5M rows.

Task 2d: Pipeline Optimization

Customer 360 pipeline takes 4 hours. Propose optimizations: identify bottlenecks, partitioning strategy (by registration_date? province?), incremental vs. full rebuild trade-offs, resource sizing for 12M customers, backfill strategy for 2 years historical data.

Question 3: Engineering Standards & Team Enablement

Technical Leadership, Documentation, Testing, Observability

Context

You inherit a team of 8 junior/mid engineers. Current state: 200+ undocumented DAGs, 40% failure rate, no code reviews, no tests, tribal knowledge, 500+ alerts/day ignored. You have 6 months to professionalize.

Tasks

Task 3a: Engineering Standards Document

Create Data Engineering Standards Guide (2-3 pages):

• Code Standards: style guide, naming conventions, documentation requirements

• Pipeline Standards: idempotency, error handling, logging, configuration

• Quality Gates: code review checklist, required tests, DQ checks

• Operational Standards: alerting philosophy, runbooks, incident response

Task 3b: Testing Strategy

Design testing strategy for data pipelines:

• Unit Tests: What to test? Provide 3 example test cases for Customer 360 pipeline

• Integration Tests: How to test end-to-end? Test data strategy?

• Data Quality Tests: What assertions in every pipeline? How to handle failures?

• Write sample pytest code for one transformation function

Task 3c: Observability & Alerting

Design observability stack:

• Metrics: pipeline health, DQ scores, infrastructure utilization

• Dashboards: executive view vs. on-call view

• Alerting: reduce 500 alerts/day to <20 actionable. Define severity levels, routing, escalation

• Sample alert definition for "Customer 360 pipeline delayed >2 hours"

Task 3d: Team Enablement Plan

Create 90-day plan:

- Week 1-4: Quick wins — what do you tackle first?
- Week 5-8: Process rollout — code reviews, testing
- Week 9-12: Training/workshops
- Knowledge transfer from 2 senior engineers with tribal knowledge
- Success metrics: target failure rate <10%, alerts <20/day

— End of Level 3 —

End of Assessment